

Queens Problem

Approccio quantistico: oracle design per l'algoritmo di Grover

1. Introduzione

Il queens problem è un classico esempio di problema di soddisfacimento di vincoli (CSP); per tale ragione, si presta molto bene ad essere implementato come oracolo per l'algoritmo di Grover, che è in grado di riconoscere le soluzioni che soddisfano vincoli complessi, anche in spazi di ricerca molto grandi (a meno di limiti fisici dei computer quantistici).

2. Scelte progettuali

Per la realizzazione della logica dell'oracolo, si è adottato in una prima fase un approccio bottom-up, ovvero si è implementato e visualizzato direttamente in Qiskit il caso più semplice (4x4) per poi astrarne delle regole per il caso generale (NxN), come quelle per il dimensionamento del circuito e per la costruzione dei suoi componenti.

Successivamente, si è utilizzato l'ambiente Prolog, basato sul framework Quantum, per l'implementazione del caso generale, in modo da passare a un approccio definitivo top-down; Prolog restituisce il circuito dell'oracolo, un componente di interfaccia lo converte in formato OpenQASM e Qiskit lo esegue utilizzando l'algoritmo di Grover.

La progettazione si declina secondo i seguenti punti:

- le righe delle regine sono fissate a-priori, ovvero l'indice di riga corrisponde a quello della regina stessa;
- lo stato è rappresentato dai valori di colonna delle regine, opportunamente codificati a seconda della dimensione della scacchiera;
- il problema è stato scomposto in tre componenti per necessità computazionali, ovvero per un compromesso tra ampiezza e profondità circuitale;
- i tre componenti sono:
 - o vincolo sulle colonne
→ $C_i \neq C_j, i, j \in [0, N-1], i \neq j$
 - o vincolo sulle diagonali differenza (o principali)
→ $R_i - C_i \neq R_j - C_j, i, j \in [0, N-1], i \neq j$
 - o vincolo sulle diagonali somma (o secondarie)
→ $R_i + C_i \neq R_j + C_j, i, j \in [0, N-1], i \neq j$
- ogni componente effettua tanti controlli di "collisione" (ovvero i casi di uguaglianza, per i quali il vincolo non è rispettato) quante sono le coppie possibili di regine ($N*(N-1)/2$);
- i controlli di collisione avvengono confrontando in ordine ciascuna regina, eccetto l'ultima, con tutte le successive;
- il numero di target qbits impiegati per memorizzare le collisioni è pari al numero di coppie possibili di regine, in quanto ogni coppia può violare solo uno dei tre vincoli;

- il vincolo sulle diagonali differenza/somma utilizza l'accumulazione di un offset positivo/negativo di colonna (con controllo di overflow), con “reset” al termine dei controlli di collisione per una data regina;
- dopo la fase di compute dei tre vincoli, viene effettuato il test di soddisfacimento dei vincoli, con inversione di fase degli stati soluzione se i vincoli sono rispettati;
- uncompute dei tre vincoli.

Di seguito si riporta lo schema circuitale dell'oracolo per le fasi di compute e test (per il caso 4x4, ma la logica rimane la stessa per NxN).

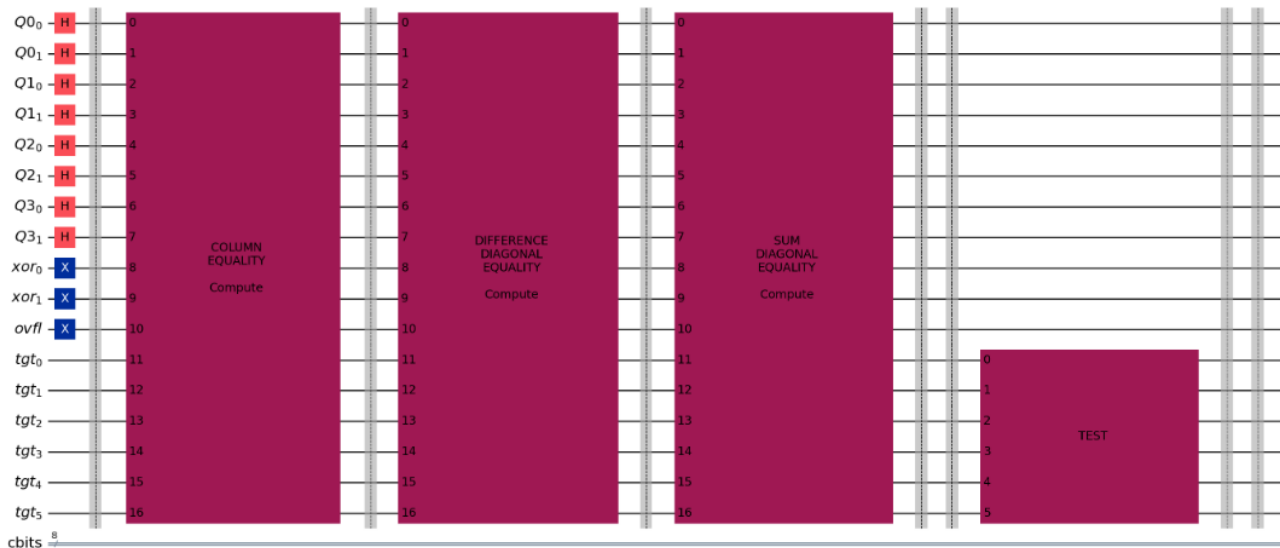


Figura 1 - Caso 4x4: schema circuitale delle fasi di compute e test

In seguito, sarà approfondita l'architettura dell'oracolo secondo il seguente iter:

- Inizializzazione
- Vincolo sulle colonne
- Vincolo sulle diagonali
- Test di soddisfacimento dei vincoli
- Composizione dell'oracolo

3. Inizializzazione

3.1. Dimensionamento del circuito

Data una scacchiera NxN, il dimensionamento del circuito è gestito dalle seguenti clausole:

- **queenDIM**: ritorno del numero di qbits necessari alla rappresentazione dello stato di una singola regina (ovvero alla codifica dei valori delle colonne), nonché, per costruzione, alla memorizzazione degli xor bitwise tra una coppia di regine ($\text{QueenDIM} = \lceil \log_2(N) \rceil$);
- **stateDIM**: ritorno del numero di qbits necessari alla rappresentazione dello stato di tutte le regine ($\text{StateDIM} = N * \text{QueenDIM}$);
- **checksDIM**: ritorno del numero di ancilla target qbits necessari a memorizzare lo stato di collisione di tutte le possibili coppie di regine ($\text{ChecksDIM} = N * (N-1) / 2$);

- **auxDIM**: ritorno del numero di ancilla qbits necessari alle operazioni intermedie delle catene Toffoli, con calcolo basato su Cmax, numero massimo di controlli presenti nelle n-tof del circuito ($AuxDIM = Cmax - 2$, con $Cmax = ChecksDIM - 1$);
- **dim**: ritorno del numero totale di qbits necessari al circuito, dato dalla somma dei precedenti qbits e da un qbit per l'overflow ($DIM = StateDIM + QueenDIM + ChecksDIM + AuxDIM + 1$);
- **get_dims**: chiamata di tutti i passi precedenti per un dato N.

Viene quindi istanziato un registro di dimensione DIM, con la seguente sequenza ordinata di qbits:

QRegister * = {State_Qbits,
XOR_Qbits,
Aux_Qbits,
Overflow_Qbit,
Target_Qbits}

* È più precisamente un'astrazione del registro; l'istanza vera e propria avviene poi in Qiskit. Anche la fase di preparazione iniziale dei qbits è delegata a Qiskit e consiste nella sovrapposizione con gate Hadamard (H) degli State_Qbits e nell'impostazione allo stato $|1\rangle$ con gate Pauli-X (X) degli XOR_Qbits e dell'Overflow_Qbit (i rimanenti qbits sono inizializzati di default allo stato $|0\rangle$).

3.2. Queens split

La lista dei qbits di stato viene trasformata, mediante la clausola **queens_split**, in una lista di sotto-liste, in cui ogni sotto-lista rappresenta lo stato di una singola regina; questa trasformazione serve a semplificare la gestione dei confronti tra le regine.

Si passa quindi dalla seguente rappresentazione:

State_Qbits = [q(0), q(1), ..., q(StateDIM-1)],

alla seguente:

Queens = [Queen(0), Queen(1), ..., Queen(N-1)],

dove il generico elemento Queen(k), con $k=[0, N-1]$, è la lista di qbits di dimensione QueenDIM relativa alla k-esima regina.

3.3. Parametrizzazione

Il registro, la lista Queens e altri parametri vengono restituiti dalla clausola **init**, per poi essere passati alle opportune clausole dei componenti della logica circuitale; tali clausole, sono approfondite nei successivi paragrafi.

4. Vincolo sulle colonne

Il vincolo sulle colonne viene realizzato con le seguenti clausole:

- **col_eq**: costruzione del circuito parziale relativo ai confronti di uguaglianza sulle colonne per una data regina con tutte le successive;
- **col_eq_compute**: chiamata di **col_eq** per ogni regina (eccetto l'ultima).

In particolare, per l'operatore di uguaglianza si sono costruiti due sotto-circuito modulari, partendo dall'osservazione che il sotto-operatore di AND reduction fosse riutilizzabile, ad eccezione dell'azione sul target del confronto; tali moduli sono realizzati dalla clausola **check_subcircuit**, che restituisce l'implementazione della catena di AND (ad esclusione dell'AND sul target) e della sua inversa. La clausola **init** si occupa di passare questi due moduli come argomenti di input a **col_eq_compute**; in **col_eq** vengono quindi congiunti all'azione sul target, diversa per ogni confronto.

4.1. Caso 4x4

Il mapping tra i qbits di Prolog e Qiskit per il caso 4x4 è il seguente:

State_Qbits: $q(0) - q(7) \leftrightarrow Q0_0 - Q3_1$

XOR_Qbits: $q(8), q(9) \leftrightarrow \text{xor}_0, \text{xor}_1$

Aux_Qbits*: $q(10) - q(12) \leftrightarrow \emptyset **$

Overflow_Qbit: $q(13) \leftrightarrow \text{ovfl}$

Target_Qbits: $q(14) - q(19) \leftrightarrow \text{tgt}_0 - \text{tgt}_5$

* Per il caso 4x4 il vincolo delle colonne non utilizza qbits ausiliari perché non sono presenti gate con più di 2 controlli.

** L'implementazione di Qiskit utilizza il default 'noancilla' per la gestione di gate con più di 2 controlli (scomposizione di Barenco).

Il flusso di operazioni relativo alla chiamata di **col_eq** per la regina **Q0** è il seguente:

1. **fanout** di Q0 su XOR_Qbits ($q(8), q(9)$);
2. **XOR bitwise** di Q0 con Q1, con memorizzazione in-place su XOR_Qbits *;
3. **AND reduction** degli XOR_Qbits con target sul primo qbit non utilizzato dei Target_Qbits ($q(14) - q(19)$).
4. uncompute dello XOR bitwise su XOR_Qbits;
5. ripetere le operazioni analoghe dal passo 2 per le regine successive (Q2, Q3);
6. uncompute del fanout di Q0 su XOR_Qbits.

* Gli XOR_Qbits sono negati in fase di inizializzazione, per cui l'operazione effettiva del passo 2 è uno XNOR bitwise.

I passi 2, 3 e 4 compongono l'operatore di uguaglianza (**EQ**).

Di seguito si riporta lo schema circuitale del flusso descritto e i corrispondenti sotto-circuiti in Qiskit e Prolog.

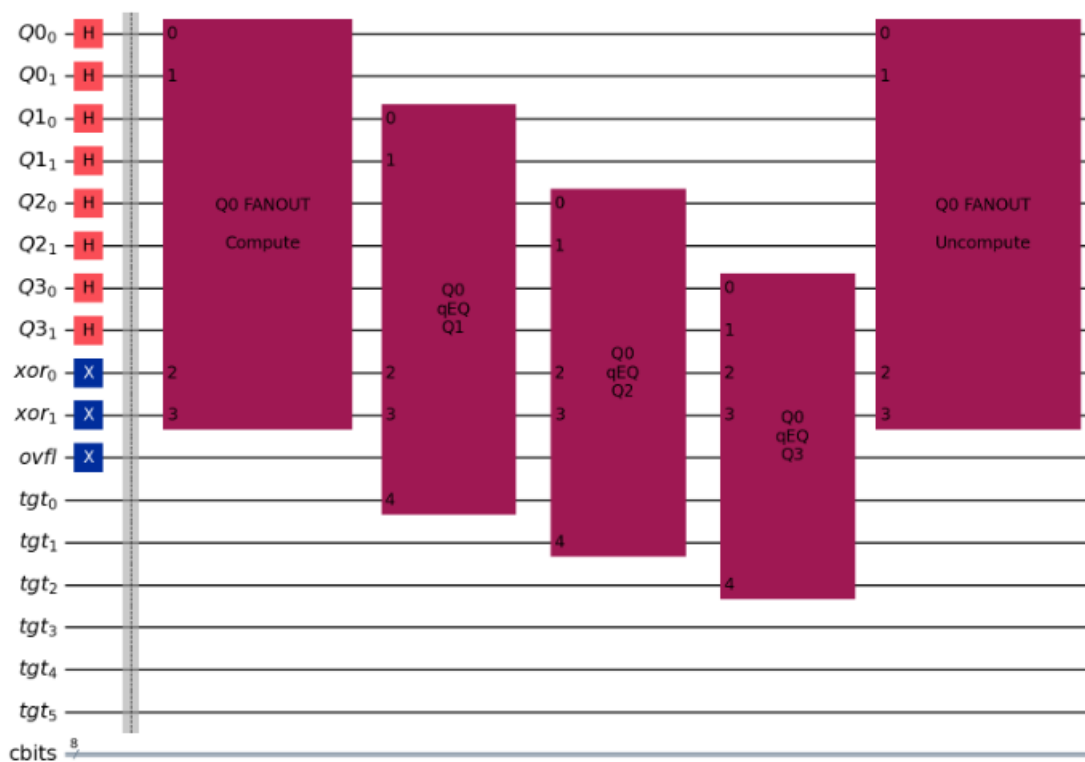


Figura 2 - Caso 4x4: vincolo sulle colonne per Q0 (schema Qiskit)

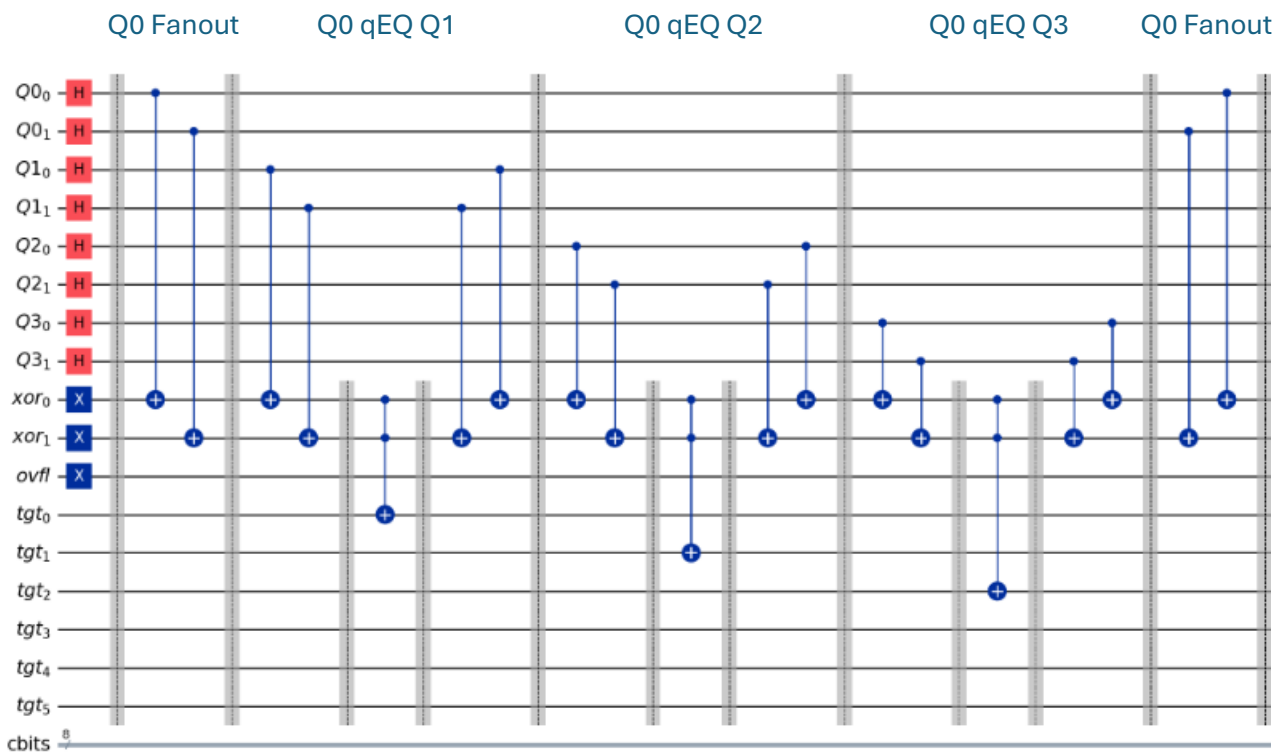


Figura 3 - Caso 4x4: vincolo sulle colonne per Q0 (Qiskit)

<code>[q(8):[[cnot(q(0),q(8))]],q(9):[[cnot(q(1),q(9))]]],</code>	<code>% Q0 FANOUT</code>	
<code>[q(8):[[cnot(q(2),q(8))]],q(9):[[cnot(q(3),q(9))]]],</code>	<code>% Q0 qXOR Q1</code>	<code>%</code>
<code>[q(14):[[tof(q(8),q(9),q(14))]]],</code>	<code>% qAND reduction</code>	<code>% Q0 qEQ Q1</code>
<code>[q(9):[[cnot(q(3),q(9))]],q(8):[[cnot(q(2),q(8))]]],</code>	<code>% Q0 qXOR Q1</code>	<code>%</code>
<code>[q(8):[[cnot(q(4),q(8))]],q(9):[[cnot(q(5),q(9))]]],</code>	<code>% Q0 qXOR Q2</code>	<code>%</code>
<code>[q(15):[[tof(q(8),q(9),q(15))]]],</code>	<code>% qAND reduction</code>	<code>% Q0 qEQ Q2</code>
<code>[q(9):[[cnot(q(5),q(9))]],q(8):[[cnot(q(4),q(8))]]],</code>	<code>% Q0 qXOR Q2</code>	<code>%</code>
<code>[q(8):[[cnot(q(6),q(8))]],q(9):[[cnot(q(7),q(9))]]],</code>	<code>% Q0 qXOR Q3</code>	<code>%</code>
<code>[q(16):[[tof(q(8),q(9),q(16))]]],</code>	<code>% qAND reduction</code>	<code>% Q0 qEQ Q3</code>
<code>[q(9):[[cnot(q(7),q(9))]],q(8):[[cnot(q(6),q(8))]]],</code>	<code>% Q0 qXOR Q3</code>	<code>%</code>
<code>[q(9):[[cnot(q(1),q(9))]],q(9):[[cnot(q(0),q(8))]]],</code>	<code>% Q0 FANOUT</code>	

Figura 4 - Caso 4x4: vincolo sulle colonne per Q0 (Prolog)

Il sotto-circuito viene ripetuto successivamente per Q1 e Q2 in modo analogo per completare la fase di compute relativa al vincolo sulle colonne.

5. Vincolo sulle diagonali

Il vincolo sulle diagonali viene realizzato con le seguenti clausole:

- **diag_eq**: costruzione del circuito parziale relativo ai confronti di uguaglianza sulle diagonali per una data regina, con accumulazione di un offset di colonna, con tutte le successive, e “reset” dell’offset accumulato al termine dei confronti;
- **diag_eq_compute**: chiamata di **diag_eq** per ogni regina (eccetto l’ultima).

Il vincolo sulle diagonali viene costruito due volte, una per le diagonali differenza (offset di colonna positivo) e una per le diagonali somma (offset di colonna negativo);

In particolare, per l’offset di colonna si sono costruiti due sotto-circuiti modulari, partendo dall’osservazione che la struttura di incremento fosse riutilizzabile; tali moduli vengono realizzati dalla clausola **incr_subcircuit**, che restituisce l’implementazione dell’incremento e del suo inverso (il decremento). La clausola **init** si occupa poi di passare questi due moduli come argomenti di input nelle due chiamate di **diag_eq_compute**, in cui si scambiano essenzialmente di ruolo come parametri.

Infine, analogamente a quanto visto per il vincolo sulle colonne, anche in questo caso viene utilizzato il modulo **check_subcircuit** per l’operatore di uguaglianza.

5.1. Caso 4x4

Il mapping tra i qbits di Prolog e Qiskit è il seguente:

State_Qbits: $q(0) - q(7) \leftrightarrow Q0_0 - Q3_1$

XOR_Qbits: $q(8), q(9) \leftrightarrow \text{xor}_0, \text{xor}_1$

Aux_Qbits: $q(10) - q(12) \leftrightarrow \emptyset *$

Overflow_Qbit: $q(13) \leftrightarrow \text{ovfl}$

Target_Qbits: $q(14) - q(19) \leftrightarrow \text{tgt}_0 - \text{tgt}_5$

* L'implementazione di Qiskit utilizza il default 'noancilla' per la gestione di gate con più di 2 controlli (scomposizione di Barenco).

Il flusso di operazioni relativo alla chiamata di **diag_eq** per la regina **Q0** nel caso del vincolo sulle diagonali differenza è il seguente:

1. **fanout** di Q0 su XOR_Qbits ($q(8), q(9)$);
2. **incremento** su XOR_Qbits e Overflow_Qbit ($q(13)$), con cui viene applicato un offset di +1 al valore di colonna di Q0, se non si attiva lo stato di overflow che invalida il confronto (valore di colonna pre-incremento pari a '11' binario);
3. **XOR bitwise** di $(Q0 + 1)$ con Q1, con memorizzazione in-place su XOR_Qbits *;
4. **AND reduction** degli XOR_Qbits ed Overflow_Qbit con target il primo qbit non utilizzato dei Target_Qbits ($q(14) - q(19)$) e ancilla ausiliaria $q(10)$ (l'operatore include l'uncompute dell'AND intermedio su $q(10)$);
5. uncompute dello XOR bitwise su XOR_Qbits;
6. ripetere le operazioni analoghe dal passo 2 per le regine successive (Q2, Q3);
7. uncompute dell'offset accumulato;
8. uncompute del fanout di Q0 su XOR_Qbits;

* Gli XOR_Qbits sono negati in fase di inizializzazione, per cui l'operazione effettiva del passo 2 è uno XNOR bitwise.

Il passo 2 utilizza il modulo di incremento, il passo 7 il modulo di decremento (l'inverso).

I passi 3, 4 e 5 compongono l'operatore di uguaglianza (**EQ**).

Di seguito si riporta lo schema circuitale del flusso descritto e i corrispondenti sotto-circuiti in Qiskit e Prolog.

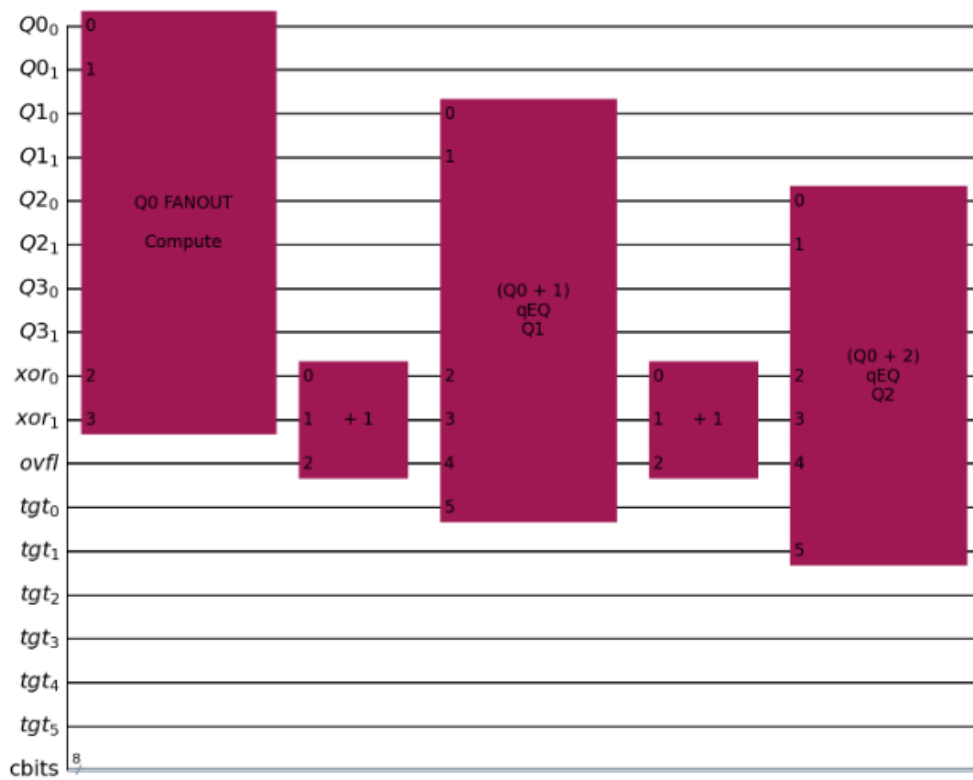


Figura 5 - Caso 4x4: vincolo sulle diagonali differenza per Q0, parte 1 (schema Qiskit)

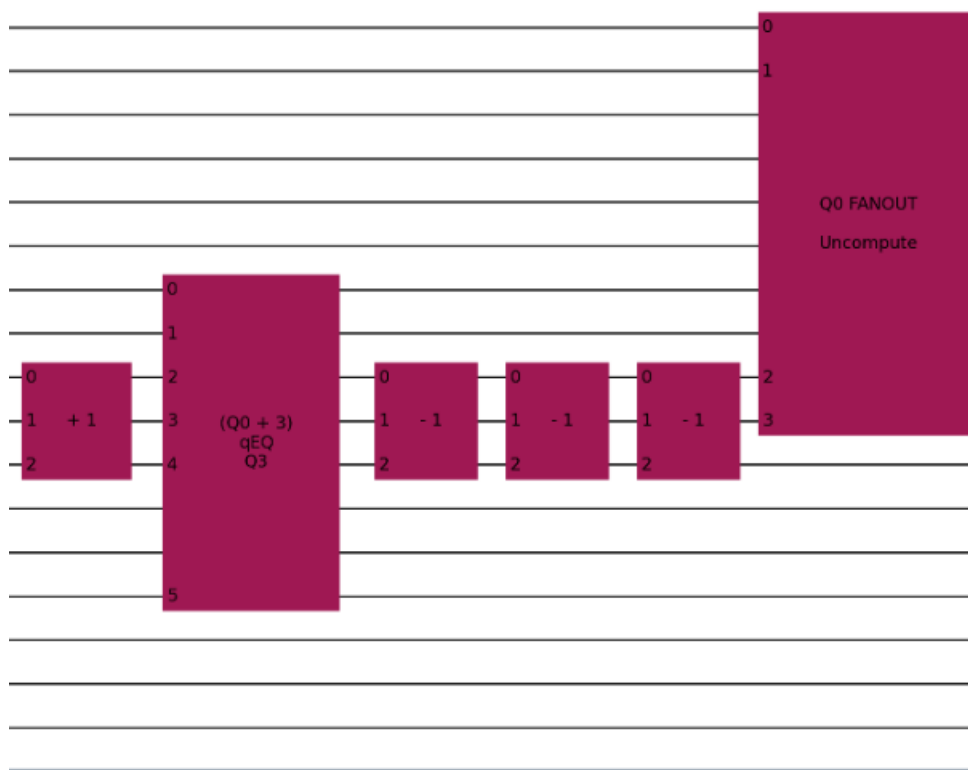


Figura 6 - Caso 4x4: vincolo sulle diagonali differenza per Q0, parte 2 (schema Qiskit)

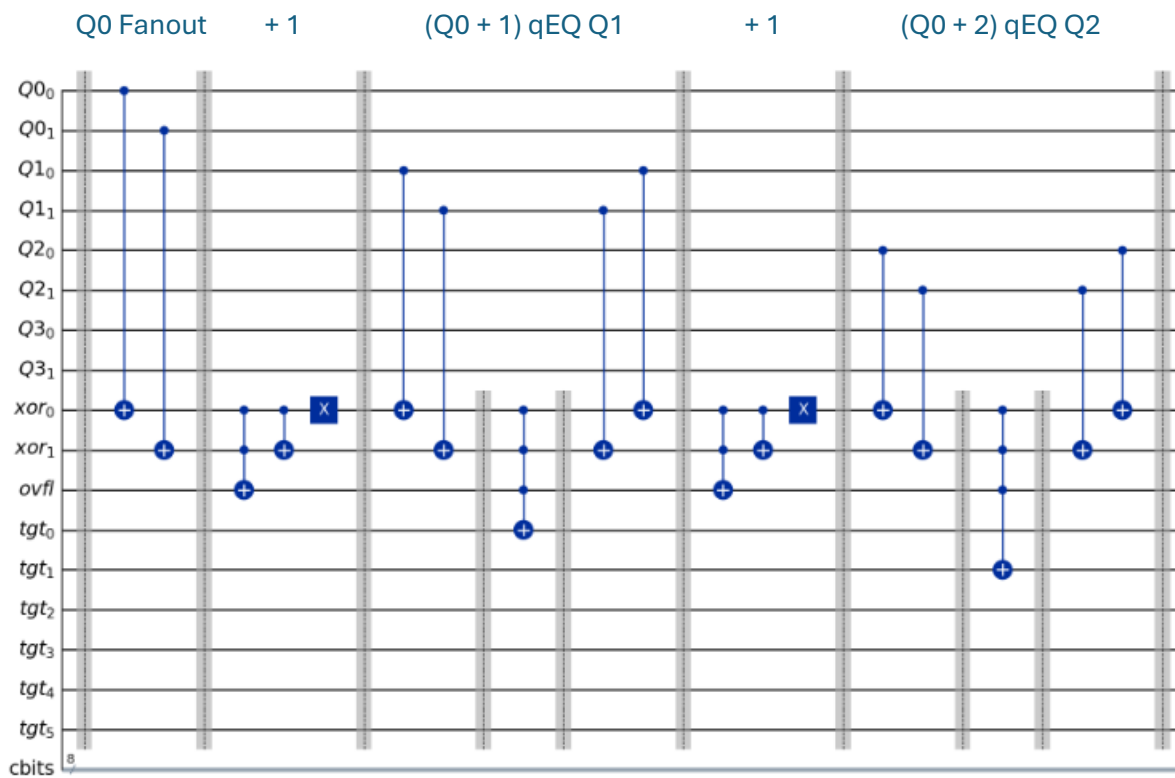


Figura 7 - Caso 4x4: vincolo sulle diagonalì differenza per Q0, parte 1 (Qiskit)

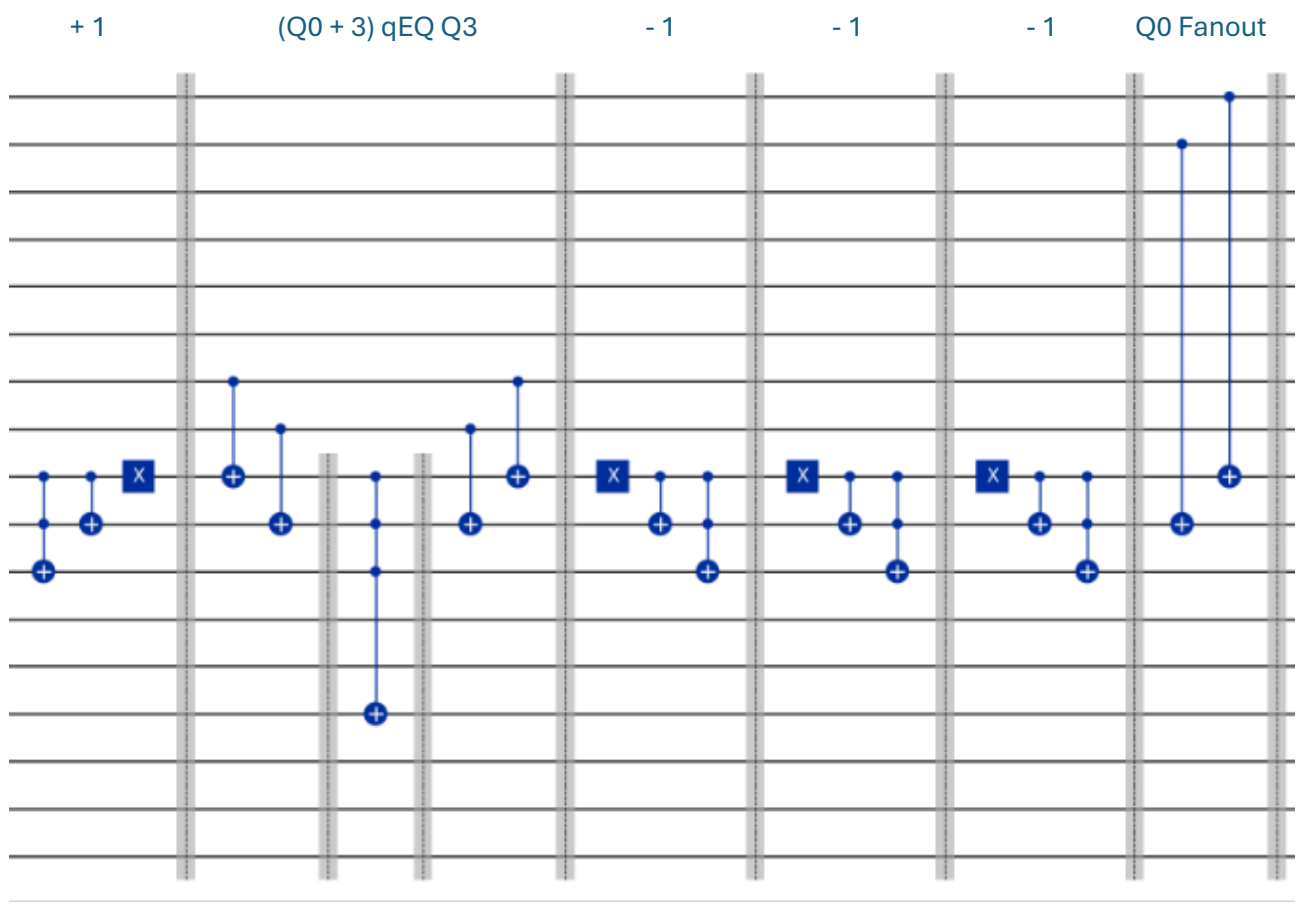


Figura 8 - Caso 4x4: vincolo sulle diagonalì differenza per Q0, parte 2 (Qiskit)

```

[q(8):[[cnot(q(0),q(8))]],q(9):[[cnot(q(1),q(9))]]],          % Q0 FANOUT

[q(13):[[tof(q(8),q(9),q(13))]]],                             %
[q(9):[[cnot(q(8),q(9))]]],                                   % + 1
[q(8):[[not(q(8))]]],                                         %

[q(8):[[cnot(q(2),q(8))]],q(9):[[cnot(q(3),q(9))]]],          % (Q0 + 1) qXOR Q1 %
[q(10):[[tof(q(8),q(9),q(10))]]],                             %
[q(14):[[tof(q(13),q(10),q(14))]]],                             % qAND reduction % (Q0 + 1) qEQ Q1
[q(10):[[tof(q(8),q(9),q(10))]]],                             %
[q(9):[[cnot(q(3),q(9))]],q(8):[[cnot(q(2),q(8))]]],          % (Q0 + 1) qXOR Q1 %

[q(13):[[tof(q(8),q(9),q(13))]]],                             %
[q(9):[[cnot(q(8),q(9))]]],                                   % + 1
[q(8):[[not(q(8))]]],                                         %

[q(8):[[cnot(q(4),q(8))]],q(9):[[cnot(q(5),q(9))]]],          % (Q0 + 2) qXOR Q2 %
[q(10):[[tof(q(8),q(9),q(10))]]],                             %
[q(15):[[tof(q(13),q(10),q(15))]]],                             % qAND reduction % (Q0 + 2) qEQ Q2
[q(10):[[tof(q(8),q(9),q(10))]]],                             %
[q(9):[[cnot(q(5),q(9))]],q(8):[[cnot(q(4),q(8))]]],          % (Q0 + 2) qXOR Q2 %

```

Figura 9 - Caso 4x4: vincolo sulle diagonali differenza per Q0, parte 1 (Prolog)

```

[q(13):[[tof(q(8),q(9),q(13))]]],                             %
[q(9):[[cnot(q(8),q(9))]]],                                   % + 1
[q(8):[[not(q(8))]]],                                         %

[q(8):[[cnot(q(6),q(8))]],q(9):[[cnot(q(7),q(9))]]],          % (Q0 + 3) qXOR Q3 %
[q(10):[[tof(q(8),q(9),q(10))]]],                             %
[q(16):[[tof(q(13),q(10),q(16))]]],                             % qAND reduction % (Q0 + 3) qEQ Q3
[q(10):[[tof(q(8),q(9),q(10))]]],                             %
[q(9):[[cnot(q(7),q(9))]],q(8):[[cnot(q(6),q(8))]]],          % (Q0 + 3) qXOR Q3 %

[q(8):[[not(q(8))]]],                                         %
[q(9):[[cnot(q(8),q(9))]]],                                   % - 1
[q(13):[[tof(q(8),q(9),q(13))]]],                             %

[q(8):[[not(q(8))]]],                                         %
[q(9):[[cnot(q(8),q(9))]]],                                   % - 1
[q(13):[[tof(q(8),q(9),q(13))]]],                             %

[q(8):[[not(q(8))]]],                                         %
[q(9):[[cnot(q(8),q(9))]]],                                   % - 1
[q(13):[[tof(q(8),q(9),q(13))]]],                             %

[q(9):[[cnot(q(1),q(9))]],q(8):[[cnot(q(0),q(8))]]],          % Q0 FANOUT

```

Figura 10 - Caso 4x4: vincolo sulle diagonali differenza per Q0, parte 2 (Prolog)

Il sotto-circuito viene ripetuto successivamente per Q1 e Q2 in modo analogo per completare la fase di compute relativa al vincolo sulle diagonali differenza.

Per il vincolo sulle diagonali somma si procede in modo analogo, ma con il ruolo invertito dei moduli di incremento e decremento.

6. Test di soddisfacimento dei vincoli

Al termine della fase di compute e prima della fase di uncompute, viene effettuato il test di soddisfacimento dei vincoli, realizzato dalla clausola **test**; se i vincoli sono rispettati, avviene l'inversione di fase degli stati soluzione.

6.1. Caso 4x4

Il mapping tra i qbits di Prolog e Qiskit è il seguente:

State_Qbits: $q(0) - q(7) \leftrightarrow Q0_0 - Q3_1$

XOR_Qbits: $q(8), q(9) \leftrightarrow xor_0, xor_1$

Aux_Qbits: $q(10) - q(12) \leftrightarrow \emptyset *$

Overflow_Qbit: $q(13) \leftrightarrow ovfl$

Target_Qbits: $q(14) - q(19) \leftrightarrow tgt_0 - tgt_5$

* L'implementazione di Qiskit utilizza il default 'noancilla' per la gestione di gate con più di 2 controlli (scomposizione di Barenco).

Il test consiste in un **NOR reduction su asse Z** dei Target_Qbits con Aux_Qbits ($q(10) - q(12)$) e H su $q(19)$ (l'operatore include uncompute di H su $q(19)$, degli AND intermedi su Aux_Qbits e dei NOT sui Target_Qbits).

Non è presente un qbit target vero e proprio, perché, spostando la base computazionale all'asse X (con H su $q(19)$), l'operazione diventa una rotazione attorno all'asse Z, e agisce dunque sulla fase, con tutti i Target_Qbits come controlli; $q(19)$ può essere visto come 'target', solo in quanto destinazione finale del NOR reduction.

Di seguito si riportano i sotto-circuiti Qiskit e Prolog relativi al test.

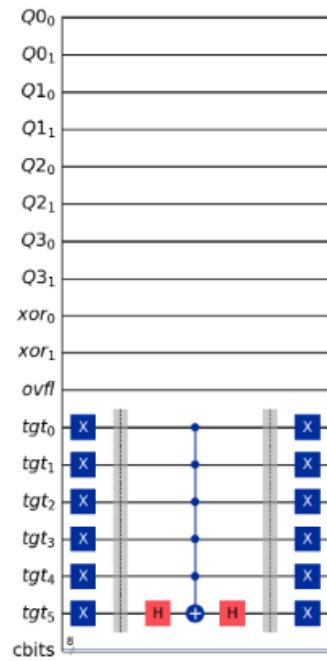


Figura 11 - Caso 4x4: test (Qiskit)

```

[q(14):[[not(q(14))]], %
q(15):[[not(q(15))]], %
q(16):[[not(q(16))]], %
q(17):[[not(q(17))]], %
q(18):[[not(q(18))]], %
q(19):[[not(q(19))]], %
[q(12):[[tof(q(14),q(15),q(12))]], %
[q(11):[[tof(q(16),q(12),q(11))]], %
[q(10):[[tof(q(17),q(11),q(10))]], %
[q(19):[[h(q(19))]], %
[q(19):[[tof(q(10),q(18),q(19))]], % Z-axis qAND reduction % Z-axis qNOR reduction
[q(19):[[h(q(19))]], %
[q(10):[[tof(q(17),q(11),q(10))]], %
[q(11):[[tof(q(16),q(12),q(11))]], %
[q(12):[[tof(q(14),q(15),q(12))]], %
[q(19):[[not(q(19))]], %
q(18):[[not(q(18))]], %
q(17):[[not(q(17))]], %
q(16):[[not(q(16))]], %
q(15):[[not(q(15))]], %
q(14):[[not(q(14))]] %

```

Figura 12 - Caso 4x4: test (Prolog)

7. Composizione dell'oracolo

Il circuito finale, ovvero l'oracolo del queens problem, viene restituito, per un dato N, dalla clausola **queens_problem**, che svolge in ordine le seguenti chiamate:

- **init**: per un dato N, restituisce tutti i parametri necessari ai componenti circuitali;
- **col_eq_compute**: restituisce il circuito compute per il vincolo sulle colonne;
- **diag_eq_compute**: restituisce il circuito compute per il vincolo sulle diagonali differenza;
- come il passo precedente, ma per il vincolo sulle diagonali somma;
- **test**: restituisce il circuito per il test.

Infine, esegue la concatenazione dei componenti e dell'uncompute.

La clausola **queens_problem** è anche l'interfaccia di utilizzo per effettuare query prolog su N arbitrari, in tre modi differenti:

- “?- queens_problem(N,C).” restituisce il circuito dell'oracolo;
- “?-queens_problem(N,C1,C2).” restituisce il circuito di compute e il circuito di test;
- “?- queens_problem(N,C1,C2,C3,C4).” restituisce i circuito di compute di ciascun vincolo e il circuito di test.